

pm4py-mcp: a phased roadmap for the first process-mining MCP server

The thesis in one sentence: pm4py-mcp should ship as an AGPL-licensed, stdio-first MCP server that wraps a deliberately small slice of PM4Py behind handle-based tools, then grows over five phases into the reference open-source substrate for agentic process mining. The opportunity is unusual: no MCP server for process mining exists yet, PM4Py already contains a `pm4py.llm` module explicitly designed to feed event logs to LLMs, and commercial incumbents (Celonis, SAP Signavio, Microsoft Power Automate) are racing to build proprietary equivalents — Microsoft even shipped its own MCP server for Power Automate Process Mining. An open, local-first, research-grade alternative filling this gap is both technically tractable and strategically defensible. Built correctly, pm4py-mcp becomes the canonical integration point between Claude (and any MCP-capable agent) and the open process mining ecosystem.

Strategic landscape and positioning

The process-mining-plus-AI space is crowded but **the open, local, Python-native quadrant is empty**. Celonis Process Copilot, SAP Joule Agents for Signavio, Apromore's four-copilot portfolio, and Microsoft's Power Automate Process Mining MCP server (Microsoft Learn) all bundle LLM features — but all are closed SaaS tied to proprietary data platforms. Academic work has moved faster: Berti, Schuster, and Kourani's 2023–2026 papers established the "**abstract-then-prompt**" paradigm now implemented in `pm4py.llm.abstract_*` (Springer) and their 2024 AgWf paper (Re-Thinking Process Mining in the AI-Based Agents Era) formalized the architecture an MCP server should implement — a tuple $\langle F, T \rangle$ of deterministic tools and non-deterministic AI tasks, (arXiv) (The Moonlight) with clean separation of duties so the LLM never computes fitness numbers itself. The **PM_Ax** preprint (March 2026) validates the split further with an Engineer/Analyst agent architecture running a whitelisted PM4Py API wrapper (ResearchGate) — essentially the proof-of-concept for what pm4py-mcp productizes.

This positions pm4py-mcp on four axes commercial MCPs cannot match: **local-first privacy** (event logs never leave the machine; works with Ollama/local models), **research-grade algorithms** (Inductive Miner variants, alignments, POWL, OCEL 2.0, object-centric Petri nets), (Processintelligence) (Processintelligence) **standards-native I/O** (XES, OCEL 2.0, BPMN, PNML), (Processintelligence) and **composability** into LangGraph / CrewAI / AutoGen crews. The Microsoft MCP offers roughly **nine tools** (Microsoft Learn) (Microsoft Learn) on a polished cloud surface — pm4py-mcp's answer is depth (hundreds of PM4Py functions, wrapped intelligently) and openness.

Foundational architecture decisions

Seven decisions cascade through every phase and should be locked before writing any code.

State is handle-based and in-memory. Event logs are 10 MB – 1 GB pandas DataFrames; Claude Desktop enforces a **~1 MB hard cap on any single tool/resource response**, (GitHub) so returning logs is physically impossible. The recommended pattern is a server-side (LogRegistry) (LRU of

~8 logs, 1-hour TTL, configurable) keyed by a short (`log_id`) handle. Tools accept either a file path (first call) or a prior handle (subsequent calls) and return a new handle plus a tiny summary. (`Apidog`) Workspace directory (`~/ .pm4py-mcp/workspace`) holds derived artifacts; state does **not** persist across client restarts, which users must be told. MCP Resources are reserved for bundled sample logs, not computation handles.

Visualizations use a dual-channel return. Every render tool saves both PNG and SVG to the workspace, returns a text summary plus absolute paths, and attaches an (`ImageContent`) block only if a downscaled PNG fits under ~700 KB — below the 1 MB response ceiling. Claude Desktop renders the inline PNG inside the collapsible tool-use accordion (`GitHub`) (not the final reply), (`GitHub`) so the text-plus-path channel is what users actually see; Claude Code (a terminal) renders nothing inline, making file paths mandatory there. SVG inside (`ImageContent`) is spec-legal but poorly supported in practice. (`Apidog`)

The SDK is FastMCP via the official (`mcp`) Python package, pinned (`mcp>=1.20, <2`) to avoid the pre-alpha v2 rewrite on (`main`). Transport is **stdio only** in v1 (zero-setup, works identically in Claude Desktop and Claude Code); Streamable HTTP becomes a v1.2 option for shared team deployments. SSE is deprecated spec-wide and should not be built. (`Apidog +3`)

OCEL 2.0 gets a parallel tool namespace, not overloading. Object-centric logs have a fundamentally different data model (events ↔ objects with qualifiers, object types, o2o relations). Overloading (`discover_petri_net`) to accept both confuses the LLM planner and doubles the parameter space. Instead, ship a parallel (`ocel_*`) surface plus a pivotal (`flatten_ocel(ocel_id, object_type) → log_id`) bridge tool that returns a traditional-log handle, so all XES-side tools compose with OCEL via an object-type perspective.

The license is AGPL-3.0-or-later, matching PM4Py's upstream. PM4Py's current license on the (`release`) branch is AGPLv3 (Process Intelligence Solutions dual-licenses commercially). (`PyPI +2`) Because (`import pm4py`) is in-process linking, any permissive license on the wrapper is effectively unenforceable and exposes maintainers to takedown risk. AGPL also correctly propagates PIS's commercial business model: enterprises bundling pm4py-mcp into closed SaaS must either open-source their combined work (`AiOw`) or contact PIS for a commercial PM4Py license. (`PyPI`) Include a DCO sign-off requirement; avoid CLAs.

Packaging uses hatchling + (`src/pm4py_mcp/`) layout + PyPI Trusted Publishing. Python 3.10–3.14 (intersection of MCP SDK 3.10+ and PM4Py 3.9–3.14). (`GitHub`) (`PyPI`) Core install pulls PM4Py (which already brings Graphviz); extras (`[ocel]`) adds (`pyarrow`) for relational OCEL, (`[llm]`) adds (`openai`) for (`pm4py.llm.openai_query`), (`Processintelligence`) (`[dev]`) adds test/lint tooling. Primary install channel is (`uvx pm4py-mcp`) — the first README snippet is the (`claude_desktop_config.json`) entry, not (`pip install`), because MCP users configure rather than install. (`Apidog`)

Tool surface stays small and workflow-shaped, not 1:1 with PM4Py. A widely cited MCP anti-pattern is "dumping 43 tools into the context window" (`Apigene`) — every tool description burns prompt budget and confuses the planner. (`Apidog`) The Motherduck MCP (one (`query`) tool), (`GitHub`) the Block Jupyter MCP (four workflow tools), and Microsoft's Process Mining MCP (nine

tools) are the reference examples. pm4py-mcp should consolidate around ~15-25 tools organized as verbs (load, describe, discover, filter, conform, visualize, stats, export) rather than exporting the entire ~200-function PM4Py surface.

Phase 0 — Foundations (1-2 weeks)

The goal of Phase 0 is a **walking skeleton** that proves end-to-end integration before any feature work. Deliverables: (1) a `pm4py-mcp` PyPI placeholder claimed via Trusted Publishing, (2) the `pyproject.toml` + hatchling build + entry point verified with `uvx pm4py-mcp@latest`, (3) a single `ping` tool reachable from Claude Desktop, Claude Code, and MCP Inspector, (4) the four-layer testing pyramid in CI (pytest unit → in-process `ClientSession` → stdio subprocess → Inspector CLI nightly) on matrix Python 3.10-3.13 × {Ubuntu, macOS, Windows}. The `LICENSE` file, `CONTRIBUTING.md` with DCO requirement, `CODE_OF_CONDUCT.md`, `SECURITY.md`, and `CHANGELOG.md` land in this phase. Effort: small, but skipping it produces the classic MCP failure mode — a server that works in isolation and breaks mysteriously inside Claude Desktop because of PATH, CWD, or transport mismatches.

Phase 1 — Core traditional-log toolkit (3-4 weeks)

Phase 1 delivers the MVP that a process mining analyst can usefully drive from Claude. It wraps the **minimum viable PM4Py slice** identified across the I/O, discovery, conformance, filtering, statistics, and visualization categories. The complete Phase 1 tool list:

I/O and workspace: `load_event_log(path, format?)` (handles XES / CSV via `format_dataframe` / Parquet via pandas), `list_workspace()`, `describe_log(log_id)`, `export_log(log_id, format, path)`. **Discovery:** `discover_dfg(log_id)`, `discover_petri_net(log_id, algorithm={inductive|heuristics|alpha}, noise_threshold)`, `Processintelligence` `discover_bpmn(log_id)`, `discover_process_tree(log_id)`. **Conformance:** `conformance_token_replay(log_id, model_id)`, `conformance_alignments(log_id, model_id, multi_processing?)` returning fitness/ `Processintelligence` precision and per-case deviations. `Processintelligence` **Filtering:** `filter_variants(log_id, top_k|ids, retain)`, `filter_time_range`, `filter_attribute_values`, `filter_case_size`, `filter_case_performance`, `Medium`. **Statistics:** `get_variants`, `get_start_end_activities`, `get_case_durations`, `get_cycle_time`. **Visualization:** `visualize_petri_net`, `visualize_dfg`, `visualize_process_tree`, `visualize_bpmn` — each saving SVG+PNG and returning the dual-channel response.

Two cross-cutting concerns matter here. Long-running operations (alignments on a 500 MB log can exceed five minutes) **must emit progress via** `ctx.report_progress`, which resets request timeouts on most clients. `Apidog` All tools **must raise exceptions rather than returning error strings** so the SDK produces proper `isError=true` responses the LLM can recover from. `Apidog` Effort: medium. By the end of Phase 1 a user should be able to say "load sepsis.xes, discover a Petri net with 0.2 noise, check conformance, show me the diagram" and get a correct end-to-end answer.

Phase 2 — OCEL 2.0 and advanced analytics (3–4 weeks)

Phase 2 is where pm4py-mcp decisively differentiates from any existing MCP. **No commercial MCP supports OCEL 2.0.** The OCEL tool namespace lands here: `load_ocel(path)` dispatching across JSON/XML/SQLite/relational formats via `pm4py.read_ocel2*`, `processintelligence` `get_ocel_summary(ocel_id)` surfacing object types and per-type event counts, `flatten_ocel(ocel_id, object_type)` `processintelligence` returning a traditional log handle (the composability bridge), `discover_ocdfg(ocel_id)`, `discover_oc_petri_net(ocel_id)`, `Processintelligence` and the `filter_ocel_*` family with the twist that filters come in `cc_*` variants operating on connected components of the object graph.

Phase 2 also adds advanced traditional-log capability: `discover_declare` / `conformance_declare`, `discover_log_skeleton` / `conformance_log_skeleton`, `arXiv` `discover_temporal_profile` `Processintelligence` / `conformance_temporal_profile`, `discover_powl` (Kourani's Partially Ordered Workflow Language, `arXiv` central to ProMoAI), `arXiv +2` and organizational mining (`discover_handover_of_work_network`, `discover_organizational_roles`, `discover_network_analysis`). `processintelligence` Dotted chart and performance spectrum visualizations land here as matplotlib-backed PNG outputs. `play_out` for simulation and the `convert_*` family (Petri net ↔ BPMN `arXiv` ↔ Process Tree `Processintelligence` ↔ POWL) round out the conversion surface. Effort: medium; the OCEL work is novel but the PM4Py API is clean.

Phase 3 — Agentic AI capabilities (4–6 weeks)

Phase 3 is where pm4py-mcp stops being "PM4Py over MCP" and becomes an **agentic process mining substrate**. The design follows Berti's AgWf paradigm and PMAx's Engineer/Analyst split: deterministic tools compute numbers, the LLM decides what to compute and narrates results, and the MCP server enforces the separation.

The core addition is the **textual abstraction layer** — direct wrappers of `pm4py.llm.abstract_*` that convert PM artifacts into token-budget-aware text: `abstract_variants`, `Processintelligence` `abstract_dfg`, `Processintelligence` `abstract_log_attributes`, `abstract_log_features`, `Processintelligence` `abstract_petri_net`, `abstract_declare`, `abstract_log_skeleton`, `abstract_temporal_profile`, `Processintelligence` `Genai4pm2024` `abstract_case`, `abstract_ocel`, `abstract_ocdfg`, `abstract_ocel_features`. `Processintelligence` Each returns `{"content": str, "approx_tokens": int, "truncated": bool}` so the agent can plan around the context budget. These are the highest-leverage tools in the whole server because they solve the fundamental problem — event logs don't fit in a context window — and they're already implemented upstream; pm4py-mcp just exposes them.

A curated **prompt library** lands here using the MCP `@mcp.prompt` primitive (user-invoked slash commands, not LLM-selected): `conformance_workflow`, `bottleneck_analysis`, `variant_exploration`, `new_log_onboarding`, `ocel_flattening_workflow`, `semantic_anomaly_scan`, `executive_summary`. Each is a templated multi-step instruction that

walks the LLM through a canonical PM pipeline. (Neo4j) The templates derive directly from Berti et al.'s 2023 "Abstractions, Scenarios, and Prompt Definitions" paper. (Springer)

Phase 3 also adds the **agentic plumbing**: (set_domain_context(text | path)) to load SOPs/glossaries (the equivalent of Celonis's Knowledge Model), (run_duckdb_sql(log_id, sql)) for scalable NL→SQL querying (arXiv) that bypasses token budgets, (semantic_anomaly_detect(log_id)) using Rebmman et al.'s T-SAD formulation (arxiv) via client-side sampling ((ctx.session.create_message)), (bottleneck_analysis) and (decision_mining) for structured RCA, and — critically — a (render_report(findings, artifacts)) tool that assembles a Markdown report with embedded artifacts the agent has accumulated. Effort: medium-large, but every piece is well-precedented in the literature.

Phase 4 — Hardening, extensibility, and community scale (ongoing)

Phase 4 transforms pm4py-mcp from a working tool into a **platform**. Streamable HTTP transport lands with a pluggable session-state backend (FastMCP 2.x's (AsyncKeyValue) interface over Redis (FastMCP) or disk) so team deployments work. (Apidog) A **sandboxed (exec_python) tool** — borrowed from PMAx — executes agent-written scripts against a whitelisted PM4Py API inside a restricted namespace (no filesystem writes outside workspace, no network), unlocking the long tail of PM4Py functions without having to wrap each one; this is the one place where "run arbitrary pm4py" is safe because imports are controlled. Connectors land ((extract_log_github), (extract_log_sap_o2c), (extract_log_camunda_workflow), (extract_log_outlook_calendar)) (processintelligence) so agents can ingest live operational data. A **Desktop Extension (.mcpb) bundle** gets published to Anthropic's curated directory for one-click install by non-technical users. (Apidog) Built-in **benchmark mode** ((run_pm_llm_benchmark)) lets users qualify any backend LLM against Berti's PM-LLM-Benchmark (arXiv) (Hugging Face) before trusting it on their data — a unique diagnostic capability. Integration recipes ship as (examples/langgraph_analyst_crew.py), (examples/crewai_engineer_analyst.py), and (examples/autogen_pm_team.py), cementing pm4py-mcp's place in the agent-framework ecosystem.

Community strategy and strategic positioning

The open-source strategy is aimed at two distinct audiences: **practitioners** who want to drop the server into Claude Desktop, and the **PM research community** who already know PM4Py and will cite and extend it. The README opens with a copy-paste (claude_desktop_config.json) snippet — not (pip install) — followed by a 30-second demo GIF of Claude discovering a model on the sepsis log. Docs site is **mkdocs-material on GitHub Pages** (lower ceremony than Sphinx for a project of this scope). Examples directory bundles (running-example.xes), a trimmed sepsis log, and (order-management.jsonocel); a (scripts/download_benchmark_logs.py) pulls full BPI Challenge logs from 4TU.ResearchData on demand rather than committing 200 MB files. CI enforces pre-commit (ruff, ruff-format, mypy) and runs pytest across the full matrix. GitHub Discussions, a (good-first-issue) campaign, and early presence in the MCP Discord accelerate community formation. Planned outreach includes a GenAI4PM workshop paper at ICPM 2026 and a demo submission to the

BPM tool track — these are the venues where Berti, Schuster, and Kourani publish, which doubles as a path to **upstream collaboration with Process Intelligence Solutions**. Getting their blessing (and a link from the PM4Py README) would be worth more than any other distribution channel.

Strategically, pm4py-mcp occupies a defensible niche: it is the only credible way for a Claude or GPT agent to do **real, reproducible, research-grade process mining on local data** without paying for Celonis or Signavio. As commercial MCPs multiply, the **open, standards-native, research-validated** alternative becomes more valuable, not less — the same dynamic that kept PM4Py itself relevant despite decades of commercial process mining products. The key risks to watch are PM4Py's AGPL enforcement posture (license review before any 1.0 release), the MCP SDK's v2 migration (pin `<2` until the dust settles), and Claude Desktop's evolving image rendering and size caps (the dual-channel text+path pattern is the defensive hedge). Executed well, pm4py-mcp is not just a wrapper — it is the opinionated reference implementation for how a serious analytics library meets the agent era.

Conclusion: what changes if this ships

Three things shift if pm4py-mcp lands as scoped. First, **Claude and any MCP-capable agent get first-class process mining** — the gap is closed and a new workflow (conversational analysis of XES/OCEL logs) becomes routine for any analyst with a desktop client. Second, the **agentic PM research agenda gets a reference platform** — PMAx, Jessen's crews, Berti's AgWf become reproducible on a common substrate instead of one-off notebooks, which accelerates the whole subfield. Third, and most consequentially, the **"abstract-then-prompt" paradigm escapes pm4py.llm's narrow Python API and becomes an interoperable protocol** — any agent framework speaking MCP gets Berti's abstractions for free, which is exactly the leverage Model Context Protocol was invented to produce. The phased roadmap above is calibrated so that Phase 1 alone (roughly six weeks total with Phase 0) delivers a genuinely useful tool, and each subsequent phase compounds the moat against the inevitable commercial-MCP wave.